Web Scraping 7,000 Pages in 3 Minutes with Python and Olostep

Scrape thousands of pages in a few minutes with the Olostep Batch API, download Markdown, and measure what actually finished.

By **Abid Ali Awan**, KDnuggets Assistant Editor on November 5, 2026 in **Python**



Scraping thousands of web pages can take hours, especially when you are collecting data for enterprise applications, market research, price monitoring, or large-scale AI systems. The challenge is not just scraping pages quickly, but making sure the process is reliable, scalable, and easy to manage.

I wanted to see how fast I could scrape 10,000 web pages using Python and the **Olostep Batch API**. In my test, **7,225 pages were processed in under 3 minutes**, which was impressive. However, completing the entire job, including scraping, downloading, and verifying the results, took 22 minutes and 26 seconds. Out of 10,000 URLs, 9,938 were successfully saved as Markdown files.

share the actual performance numbers, including what worked well and where the process slowed down.

Understanding How the Olostep Batch API Works

The Olostep Batch API lets you scrape up to 10,000 URLs in a single batch instead of sending thousands of individual requests. It handles the processing while you track progress, retrieve completed results, and download the extracted content.

Olostep is also expanding beyond developer-focused scraping tools to provide **web data infrastructure for enterprises**. According to its co-founder, the platform can process 100,000 pages in under 25 minutes for enterprise workloads. This kind of scale is useful for companies that regularly collect large amounts of web data for market research, competitor monitoring, product intelligence, and AI applications.

For this tutorial, I used the Olostep Python SDK to scrape **10,000 MDN documentation pages** and save their content as Markdown files. The workflow involves five steps:

1. **Prepare the URLs:** Download the MDN sitemap and select 10,000 unique pages.
2. **Submit the batch:** Send all URLs to Olostep in a single job.
3. **Track progress:** Monitor completed pages until scraping finishes.
4. **Download the results:** Retrieve the processed content and save it as Markdown.
5. **Verify the files:** Check the saved files, identify missing results, and measure the total execution time.

The workflow diagram below shows the complete process, from collecting URLs to verifying the final output.



One important thing to understand is that **a completed batch does not necessarily mean every page was successfully saved**. In my test, 9,938 out of 10,000 pages were saved, with 62 missing. This is why verifying the results matters just as much as scraping speed, especially when building reliable data pipelines.

Scraping 10,000 Pages with the Olostep Batch API

We will use the Olostep Batch API to scrape 10,000 MDN documentation pages, download their content as Markdown files, and verify how many pages were successfully collected.

Blog
Top Posts
About
Topics
AI
Career Advice
AI Tools Reviews
Data Engineering
Data Science
Language Models

8 LLM Inference Frameworks You Should Try in 2026
Web Scraping 7,000 Pages in 3 Minutes with Python and Olostep
5 Docker Containers You Need for a Home AI Lab
5 AI Certifications Worth Paying For in 2026 (And Which to Skip)
What I Did with Hermes Agent, Ollama & Local Models
5 Best GitHub Repositories for AI Agent Skills
Running LLMs Locally Does Not Mean Your Data Is Safe: What Cohere's Study Reveals
Vibe Coding a Classic Game: Building Breakout Step by Step with an AI Assistant
Which Local LMs are Best For Which Tasks

Get the **FREE ebook 'KDnuggets Artificial Intelligence Pocket Dictionary'** along with the leading newsletter on Data Science, Machine Learning, AI & Analytics straight to your inbox.

Your Email

SIGN UP

By subscribing you accept KDnuggets Privacy Policy

an Olostep API key to follow along. You can either set it as an environment variable (`OLOSTEP_API_KEY`) or enter it securely when prompted.

Note: Submitting a new batch consumes API credits, so make sure your account has sufficient credits before running the code.

// 1. Install the SDK

Start by opening a new **Jupyter Notebook** or launching the notebook provided with this tutorial. In the first cell, install the Olostep Python SDK, along with **requests** for fetching the MDN sitemap and **tqdm** for tracking scraping progress.

```
%pip -q install olostep==1.2.2 requests tqdm
```

// 2. Configure the Client

Next, we will set up the Olostep client using our API key, create a folder to store the scraped data, and define a few helper functions to save and validate the results. We will also add a small workaround for a response-handling issue in SDK version 1.2.2 to make sure we can check the batch status without errors.

```
import asyncio, csv, gzip, hashlib, json, os, random, time
from datetime import datetime, timezone
from getpass import getpass
from pathlib import Path
from xml.etree import ElementTree as ET

import requests
from olostep import AsyncOlostep
from olostep.errors import OlostepClientError_ResponseValidationFailed
from olostep.models.response import BatchInfoResponse
from tqdm import tqdm

N = 10000
MAX_WAIT_SECONDS = 2400
DATA = Path("data"); DATA.mkdir(exist_ok=True)
client = AsyncOlostep(api_key=os.getenv("OLOSTEP_API_KEY") or getpass("Olostep


def write_json(path, value):
    path.write_text(json.dumps(value, indent=2), encoding="utf-8")


def valid_file(path):
    try:
        return path.is_file() and bool(path.read_text(encoding="utf-8").strip()
    except (OSError, UnicodeError):
        return False


async def batch_info(batch_id):
    try:
        return await client.batches.info(batch_id)
    except OlostepClientError_ResponseValidationFailed as error:
        return BatchInfoResponse.model_validate({"parser": "none", **json.loads
```

// 3. Build the URL Dataset

Now, we will download the MDN sitemap and randomly select 10,000 unique documentation pages for our experiment.

The code saves these URLs in a CSV file so we can reuse the same list without downloading it again. It also checks for duplicates and makes sure we are working with the correct dataset.

```
    payload = response.content
    xml = gzip.decompress(payload) if payload.startswith(b"\x1f\x8b") else payl
    (DATA / "mdn_sitemap.xml").write_bytes(xml)
    ns = {"s": "http://www.sitemaps.org/schemas/sitemap/0.9"}
    nodes = ET.fromstring(xml).findall("s:url", ns)
    urls = sorted({node.findtext("s:loc", namespaces=ns) for node in nodes})
    urls = [url for url in urls if url and url.startswith("https://developer.mo
    random.Random(42).shuffle(urls)
    assert len(urls) >= N, f"Only {len(urls)} unique documentation URLs availab
    with (DATA / "urls.csv").open("w", newline="", encoding="utf-8") as f:
        writer = csv.writer(f); writer.writerow(["url"]); writer.writerows([url
    write_json(DATA / "dataset.json", {"source": source, "downloaded_utc": date
                                       "available_urls": len(urls), "selected_url

with (DATA / "urls.csv").open(encoding="utf-8", newline="") as f:
    urls = [row["url"] for row in csv.DictReader(f)]

assert len(urls) == len(set(urls)) == N
fingerprint = hashlib.sha256("\n".join(urls).encode()).hexdigest()
expected = {f"{i:05d}": url for i, url in enumerate(urls)}
print(f"Ready: {len(urls):,} unique URLs; source: {source}")
```

Output:

```
Ready: 10,000 unique URLs; source: https://developer.mozilla.org/sitemaps/en-us
```

The output confirms that our dataset contains 10,000 unique URLs, ready for batch scraping.

// 4. Submit and Monitor the Batch

Next, we will submit all 10,000 URLs to Olostep in a single batch and monitor the scraping progress using a progress bar.

The code also saves the batch ID, allowing us to resume the same job instead of starting over if the process gets interrupted.

```
checkpoint = Path("batch.json")
resumed = checkpoint.exists()

if resumed:
    run = json.loads(checkpoint.read_text(encoding="utf-8"))
    assert run["urls_sha256"] == fingerprint, "URL list changed; preserve the c
else:
    started = time.time()
    batch = await client.batches.create(urls=[{"url": url, "custom_id": key} fo
    run = {"batch_id": batch.id, "requested": N, "urls_sha256": fingerprint, "s
    write_json(checkpoint, run)

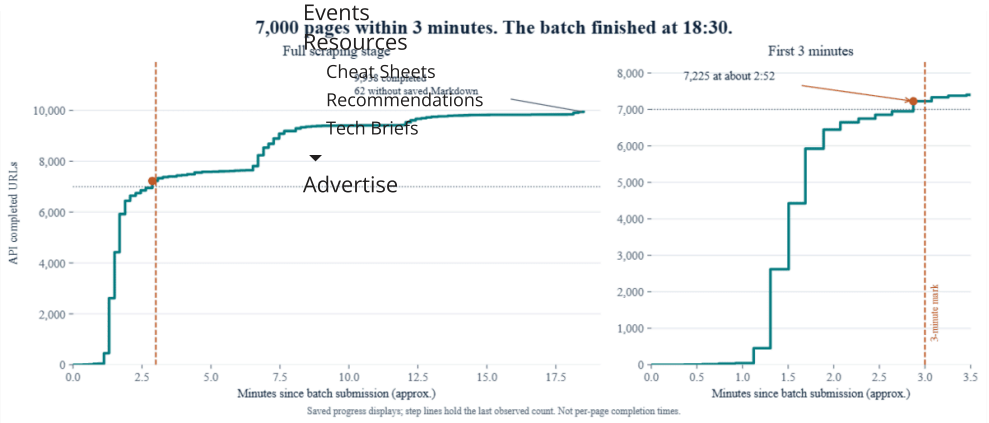
OUT = Path("output") / run["batch_id"]; OUT.mkdir(parents=True, exist_ok=True)
print("Resuming" if resumed else "Submitted", run["batch_id"])
wait_started = time.monotonic()

with tqdm(total=N, desc="Scraping", unit="page") as bar:
    while True:
        info = await batch_info(run["batch_id"])
        bar.update(max(0, info.completed_urls - bar.n))
        write_json(OUT / "progress.json", {"phase": "scraping", "status": info.
                   "processed": info.completed_urls, "total": N, "elapsed_secon
        if info.status in {"completed", "failed"}:
            break
        if time.monotonic() - wait_started > MAX_WAIT_SECONDS:
            raise TimeoutError("Batch still running. Rerun this cell to continu
        await asyncio.sleep(10)

run.setdefault("scrape_seconds", time.time() - run["started_wall"])
write_json(checkpoint, run)
print(f"Scraping stage finished: {info.status}; {info.completed_urls:,}/{N:,} c
```

KDnuggets
Machine Learning
MLOps
NLP
Programming
Python
SQL
Datasets
Events
Resources
Cheat Sheets
Recommendations
Tech Briefs
Advertise
JOIN NEWSLETTER

and 30 seconds. This shows that the initial scraping speed was impressive, but the remaining pages took considerably longer to process.



// 5. Download the Markdown

Once scraping finishes, we will retrieve the completed pages and save their content as individual Markdown files.

To speed up downloading, the code handles up to 96 retrieval requests concurrently. It also skips files that have already been saved, making it easier to resume an interrupted download.

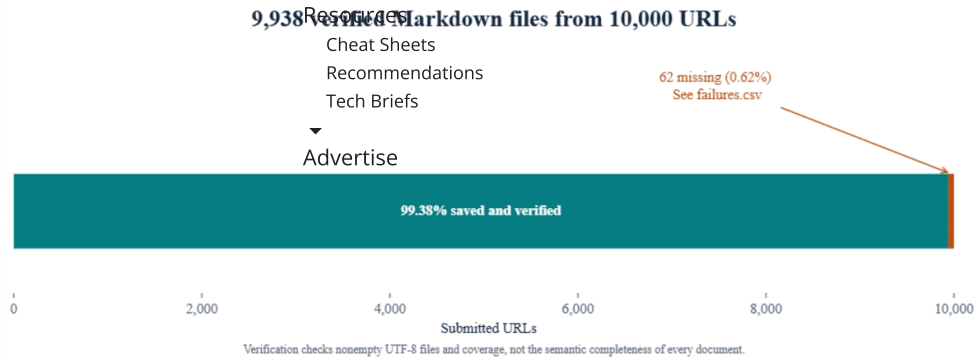
```
download_started = time.perf_counter()
limit = asyncio.Semaphore(96)
errors, scheduled, tasks = {}, set(), []
saved = set()
bar = tqdm(total=N, desc="Markdown saved", unit="file")


async def save_markdown(item):
    key = item.custom_id
    if key not in expected:
        return
    async with limit:
        path = OUT / f"{key}.md"
        if valid_file(path):
            saved.add(key); bar.update(1)
            return
        for attempt in range(2):
            try:
                async with asyncio.timeout(40):
                    content = (await item.retrieve(["markdown"])).markdown_cont
                if not content or not content.strip():
                    errors[key] = "No Markdown content returned"
                    return
                path.write_text(content, encoding="utf-8")
                saved.add(key); bar.update(1)
                if len(saved) % 50 == 0:
                    write_json(OUT / "progress.json", {"phase": "downloading",
                               "elapsed_seconds": round(time.time() - run["star
                return
            except Exception as error:
                errors[key] = str(error)
                if attempt == 0:
                    await asyncio.sleep(1)

async for item in client.batches.items(run["batch_id"], status="completed", bat
    if item.retrieve_id and item.custom_id not in scheduled:
        scheduled.add(item.custom_id)
        tasks.append(asyncio.create_task(save_markdown(item)))
await asyncio.gather(*tasks)
```

Blog
Top Posts
About
Topics
AI
Career Advice
Computer Vision
Data Engineering
Data Science
Language Models

Output: We successfully downloaded 9,938 Markdown files in approximately 2 minutes and 25 seconds after the scraping stage finished.



// 6. Verify the Results

Finally, we will check how many of the original 10,000 URLs produced valid Markdown files.

The code creates a summary of the scraping results, a CSV file linking each URL to its saved file, and a separate report for missing pages. It also calculates the total time taken, including scraping, downloading, and verification.

```
missing = {key for key in expected if not valid_file(OUT / f"{key}.md")}

with (OUT / "manifest.csv").open("w", newline="", encoding="utf-8") as f:
    writer = csv.writer(f); writer.writerow(["file", "url", "saved"])
    writer.writerows((f"{key}.md", url, key not in missing) for key, url in exp

with (OUT / "failures.csv").open("w", newline="", encoding="utf-8") as f:
    writer = csv.writer(f); writer.writerow(["url", "error"])
    writer.writerows((expected[key], errors.get(key, "Missing result or failed

total_seconds = time.time() - run["started_wall"]
summary = {"batch_id": run["batch_id"], "requested": N, "batch_status": info.st
           "api_completed_urls": info.completed_urls, "saved_markdown": N - len
           "missing_markdown": len(missing), "all_saved": not missing, "resumed
           "scrape_seconds": round(run["scrape_seconds"], 2), "download_seconds
           "total_seconds": round(total_seconds, 2),
           "all_saved_under_5_minutes": not resumed and not missing and total_s

write_json(OUT / "summary.json", summary)
write_json(OUT / "progress.json", {"phase": "finished", **summary})
print(json.dumps(summary, indent=2))
print("Files:", OUT.resolve())
print("All 10,000 Markdown files saved." if not missing else f"{len(missing):,}
await client.close()
```

The results show that 9,938 out of 10,000 pages (99.38%) were successfully saved, while 62 were missing. The complete workflow took 22 minutes and 26 seconds, which is significantly longer than the initial three-minute scraping milestone.

This final check is important because a completed batch does not necessarily mean every requested page was successfully collected.

Final Thoughts

I tested the Olostep Batch API with 1,000, 5,000, and 10,000 URLs, and what surprised me most was how much the scraping time varied between runs. My 5,000-URL batch took

Blog
Top Posts
About
Topics
AI
Career Advice
Computer Vision
Data Engineering
Data Science
Language Models

submission to final verification.

The most interesting result was the initial speed. Olostep processed over 7,000 pages in under 3 minutes, but the remaining pages took considerably longer. My guess is that the platform might dynamically allocate workers to handle large batches, although I cannot confirm how its infrastructure works internally. Slower pages, retries, or other processing delays could also explain the difference.

Overall, setting up Olostep with Python was surprisingly simple. Most of my time went into benchmarking different batch sizes, tracking progress, and validating the downloaded files rather than writing the scraping code.

The biggest takeaway is that **batch scraping can deliver impressive initial throughput, but you should measure the entire workflow rather than judging performance by the first few minutes**. In my test, 9,938 out of 10,000 URLs produced nonempty Markdown files, giving me 99.38% file coverage. For enterprise data pipelines, that visibility into both performance and missing results is just as important as scraping speed.

Abid Ali Awan (@1abidaliawan) is a certified data scientist professional who loves building machine learning models. Currently, he is focusing on content creation and writing technical blogs on machine learning and data science technologies. Abid holds a Master's degree in technology management and a bachelor's degree in telecommunication engineering. His vision is to build an AI product using a graph neural network for students struggling with mental illness.

More On This Topic

- The Best Web Scraping APIs for AI Models in 2026
- Top 7 AI Web Scraping Tools
- Best Web Scraping Companies in 2025
- Web Scraping Fundamentals for Data Science
- How to Crawl an Entire Documentation Site with Olostep
- The Best Proxy Providers for Large-Scale Scraping for 2026

Get the **FREE ebook 'KDnuggets Artificial Intelligence Pocket Dictionary'** along with the leading newsletter on Data Science, Machine Learning, AI & Analytics straight to your inbox.

Your Email
SIGN UP

By subscribing you accept KDnuggets Privacy Policy

© 2026 Guiding Tech Media | About | Contact | Advertise | Privacy | Terms of Service